

배포 환경 구축 및 트러블슈팅 경험

1. 인프라 복잡성 및 동적 IP 변경으로 인한 라우팅 문제

[Problem] 마주한 문제 및 고민

- MSA(Microservices Architecture) 환경에서 각 서비스가 독립적으로 배포될 때, 오토스케일링(Auto-scaling)으로 인해 컨테이너의 IP 주소가 동적으로 변경되는 문제가 발생했습니다.
- 이로 인해 인증/인가 서비스(API Gateway)가 뒷단 서비스로 요청을 라우팅할 때마다 네트워크 설정을 수동으로 업데이트해야 하는 번거로움과 통신 불안정성이 있었습니다.

[How To / Action] 해결 과정

- AWS ECS와 AWS Fargate를 도입하여 트래픽 증가에 유연하게 대응할 수 있는 컨테이너 기반의 표준화된 배포 플랫폼을 구축했습니다.
- 특히, **ECS Service Connect**를 활용하여 프록시 컨테이너 기반의 DNS 서비스 디스커버리(Service Discovery) 기능을 적용했습니다.

[Result] 배운 점과 느낀 점

- 동적인 IP 변경에도 자동화된 연결 관리가 가능해져 시스템의 가장 중요한 목표인 보수적 안정성을 크게 향상시킬 수 있었습니다.
- 스케일아웃 시 자동으로 로드 밸런싱이 수행되어 트래픽 집중 시의 서비스 불안정을 해소했습니다.

2. 분산 환경에서의 디버깅 병목 및 에러 추적의 한계

[Problem] 마주한 문제 및 고민

- MSA로 분리된 여러 서비스가 동시에 배포 및 운영되면서, 특정 기능에서 에러가 발생했을 때 어느 컨테이너에서 문제가 생겼는지 파악하기 어려웠습니다.

[How To / Action] 해결 과정

- **Fluent Bit**를 사이드카(Sidecar) 컨테이너 패턴으로 배치하여, 모든 서비스의 로그를 **AWS OpenSearch**로 직접 수집하도록 구성했습니다.
- Kibana 대시보드를 연동하고, 에러 로그 발생 시 즉각적으로 **Slack** 알림이 전송되도록 구현했습니다.

[Result] 배운 점과 느낀 점

- 에러 발생 시 즉각적인 인지와 원인 파악이 가능해져 디버깅 시간을 획기적으로 단축할 수 있었습니다.

3. 배포 전 로컬 환경 연동 한계와 ngrok 터널링 도입

[Problem] 마주한 문제 및 고민

- 로컬 환경(localhost)에서는 외부 API 콜백 수신이나 프론트엔드와의 사전 통합 테스트가 불가능하여, 매번 인프라에 배포한 뒤 테스트해야 하는 병목이 있었습니다.

[How To / Action] 해결 과정

- 로컬 개발 단계에서 **ngrok**을 도입하여 터널링을 통해 로컬 서버를 외부 인터넷에 임시로 노출시켰습니다.
- 특히, **Cloudflare**에 배포된 프론트엔드 환경에서 특정 도메인을 **Base URL**로 설정하여 원활한 사전 통신 테스트 환경을 구축했습니다.

[데이터 통신 흐름 예시]

1. 프론트엔드 (Cloudflare 환경)

↓ (API 요청)

2. <https://magda-semianimated-idyllically.ngrok-free.dev> (ngrok URL)

↓ (터널링 라우팅)

3. <http://localhost:8000> (로컬 백엔드 서버)

[Result] 배운 점과 느낀 점

- 실제 클라우드 환경 배포 전에 로컬에서 발생할 수 있는 연동 이슈를 사전 차단하여 개발 생산성을 크게 높일 수 있었습니다.

4. Cloudflare 기반 수동 배포 및 터널링: '빌드'와 '배포'의 분리

[Problem] 마주한 문제 및 고민

- 프론트엔드 환경을 배포하고 연결(터널링)하는 과정에서, 단순히 코드만 업로드하면 동작할 것이라는 오해가 있었습니다.
- 어떤 명령어와 절차를 거쳐야 내 로컬의 코드가 실제 사용자에게 안전하게 도달하는지 그 흐름이 명확하지 않아 잦은 배포 오류를 겪었습니다.

[How To / Action] 해결 과정

- 배포 과정이 '포장(Build)'과 '보내기(Deploy/Tunneling)'라는 두 가지 독립적인 단계로 이루어짐을 학습하고 파이프라인에 적용했습니다.
- **1단계: 포장(Build)** - 코드 작성 후 빌드 명령어를 통해 브라우저가 읽을 수 있는 정적 파일(HTML/CSS/JS)로 압축 및 최적화하는 과정을 거쳤습니다.
- **2단계: 보내기(Deploy)** - 빌드된 결과물을 CLI 환경에서 **Cloudflare** 망에 업로드하거나, 필요 시 터널을 이용해 안전하게 외부로 노출시켰습니다.

[Result] 배운 점과 느낀 점

- 원시 코드가 그대로 배포되는 것이 아니라, 통제된 환경에서 최적화된 결과물만 서버에 올라간다는 웹 생태계의 본질을 깊이 이해하게 되었습니다.
- 이러한 단계의 명확한 분리를 통해 예상치 못한 런타임 에러를 줄이고, 인프라의 보수적 안정성을 한층 더 굳건하게 다질 수 있었습니다.

요약 (포트폴리오 스펙 정리용)

구분	주요 내용
인프라 설계 목표	MSA 기반 클라우드 환경에서 보수적인 시스템 안정성을 최우선으로 담보하는 아키텍처 및 배포 환경 구축

구분	주요 내용
활용 기술 스택	AWS (ECS, Fargate, OpenSearch), Fluent Bit, ngrok, Cloudflare (Pages, Tunnel)
핵심 성과 (Result)	• ECS Service Connect를 활용한 네트워크 복잡성 추상화 및 로드밸런싱 자동화 • 사이드카 패턴(Fluent Bit) 기반 중앙화 로깅으로 디버깅 병목 해결 • ngrok 및 Cloudflare 툴을 활용한 빌드/배포 환경 분리로 운영 안정성 극대화

4. "Network Error"의 원인과 해결법

프론트엔드 환경변수(.env) 업데이트

- 프론트엔드 코드의 API Base URL이 여전히 이전 ngrok 주소로 되어 있을 가능성이 매우 높습니다.
- 프론트엔드의 .env 파일이나 API 설정 파일에서 REACT_APP_API_URL (또는 해당하는 변수명)을 방금 새로 발급받은 ngrok 주소로 변경해 주세요. (변경 후 프론트엔드 서버도 재시작해야 적용됩니다.)

소셜 로그인(OAuth) Redirect URI 수정 (해당하는 경우)

- 만약 카카오, 구글, 네이버 로그인을 연동해 두셨다면, 각 개발자 센터 콘솔에 등록된 Redirect URI가 옛날 ngrok 주소로 남아있을 것입니다.
- 로그인 후 콜백을 받을 수 없어서 에러가 발생하므로, 콘솔에 접속해 새로운 주소로 갱신해 주셔야 합니다.

백엔드 CORS 허용 목록(Allowed Origins) 확인

- 백엔드(FastAPI/Spring 등) 설정 파일에 프론트엔드나 ngrok 주소를 화이트리스트(CORS)로 등록해 두었다면, 이 부분도 새 주소로 업데이트해 주어야 합니다.

[Problem] 구체화된 에러 상황 추가

- 로컬 환경과 프론트엔드를 연동하는 과정에서, 통신 주소 불일치로 인한 `net::ERR_FAILED` 및 `net::ERR_NAME_NOT_RESOLVED` 에러가 빈번하게 발생했습니다.
- 특히, 서버 재시작 시 동적으로 변경되는 터널링 URL로 인해 프론트엔드 API 호출 시 404 에러와 `CORS Policy Blocked` 문제가 발생하여 디버깅에 많은 시간을 할애했습니다.

[How To / Action] 구체화된 해결 과정 추가

- 터널링 URL 변경 시 발생할 수 있는 휴먼 에러를 방지하기 위해 프론트엔드의 `.env` 환경 변수 관리 파이프라인을 재정비하고, 백엔드의 `CORS_ALLOWED_ORIGINS` 설정을 유연하게 대응하도록 구조화했습니다.
- 궁극적으로는 주소가 매번 변하는 `ngrok`의 한계를 극복하기 위해, 고정된 도메인을 사용할 수 있는 **Cloudflare Tunnel**로 전환하여 통신 오류를 원천 차단했습니다.

Cloudflare 배포 과정에서의 트러블슈팅 경험

1. 배경

Lumière 프로젝트를 실제 사용자에게 보여줄 수 있도록 프론트엔드와 백엔드를 배포하고, 외부에서 접근 가능한 서비스 형태로 연결하는 작업을 진행했다. 로컬 환경에서는 기능 단위로 동작을 확인할 수 있었지만, 배포 환경에서는 단순히 코드를 올리는 것만으로는 서비스가 정상적으로 동작하지 않았다.

특히 Cloudflare 배포 과정에서 외부 노출 URL, 라우팅 설정, API 요청 실패, 환경변수 반영 여부, 백엔드 연결 상태 등을 함께 확인해야 했다. 이 과정에서 “배포가 되었다”는 상태와 “사용자가 실제로 접속해 정상적으로 서비스를 사용할 수 있다”는 상태가 다르다는 점을 체감했다.

2. 마주한 문제 1: 배포는 되었지만 외부에 노출할 링크 구조를 이해하기 어려웠다

문제 상황

Cloudflare 배포 후 기본적으로 제공되는 `workers.dev` 주소는 확인할 수 있었지만, 이 주소가 실제 외부에 공개할 서비스 주소인지, 별도의 `route`나 `custom domain`을 연결해야 하는지 헷갈렸다. 배포 자체는 완료된 것처럼 보였지만, 사용자가 접속할 최종 URL을 어디서 확인하고

어떻게 관리해야 하는지 명확하지 않았다.

처음에는 “배포가 되었으니 바로 외부 링크가 만들어질 것”이라고 생각했지만, 실제로는 배포된 **Worker** 또는 프론트엔드 리소스가 어떤 도메인과 경로에 연결되는지 별도로 확인해야 했다.

어떻게 해결했는가

먼저 배포 상태와 외부 공개 상태를 분리해서 확인했다.
배포 로그에서 빌드와 업로드가 정상적으로 끝났는지 확인하고, **Cloudflare**에서 제공하는 기본 **URL**로 접속이 가능한지 확인했다. 이후 이 기본 **URL**이 테스트용 주소인지, 실제 사용자에게 공유할 주소인지 구분했다.

그 다음 **Cloudflare**의 **route**, **custom domain**, **workers.dev** 도메인의 역할을 나누어 정리했다.
workers.dev는 **Cloudflare**가 기본으로 제공하는 실행 주소이고, 실제 서비스용 주소를 만들기 위해서는 별도의 도메인 연결 또는 라우팅 설정이 필요하다는 점을 이해했다.

결과

이 문제를 통해 배포 **URL**을 단순히 “링크 하나”로 보는 것이 아니라,
배포된 리소스 → 실행 주소 → 외부 공개 주소 → 사용자 접근 경로로 나누어 이해할 수 있게 되었다.

또한 포트폴리오나 문서화 단계에서 단순히 “**Cloudflare**에 배포했다”고 쓰는 것보다, 어떤 주소가 테스트용이고 어떤 주소가 사용자 공개용인지 구분해 적어야 한다는 점을 배웠다.

3. 마주한 문제 2: **Cloudflare** 배포 중 **API**/버전 생성 오류가 발생했다

문제 상황

Cloudflare 배포 과정에서 **/workers/scripts/.../versions**와 관련된 **API** 요청 실패 메시지가 발생했다. 로그상으로는 **Cloudflare API** 요청이 실패했다는 내용이 보였지만, 처음에는 이것이 코드 문제인지, 계정 권한 문제인지, 스크립트 설정 문제인지, 배포 타겟 문제인지 바로 구분하기 어려웠다.

특히 배포 로그에는 오류가 표시되지만, 프론트 화면에서는 단순히 서비스가 정상적으로 보이지 않는 형태로 나타나기 때문에 문제의 출발점을 찾는 데 시간이 걸렸다.

어떻게 해결했는가

가장 먼저 오류가 발생한 위치를 “서비스 코드 실행 중”이 아니라 “**Cloudflare** 배포 파이프라인 내부”로 분리해서 바라보았다.
즉, 브라우저에서 발생한 오류인지, 프론트 코드 빌드 중 발생한 오류인지, **Cloudflare**가 **Worker**

버전을 생성하는 과정에서 발생한 오류인지 나누어 확인했다.

이후 배포 로그를 기준으로 다음 항목을 순서대로 점검했다.

1. Cloudflare 계정과 프로젝트가 올바르게 연결되어 있는지
2. 배포 대상 Worker 또는 Pages 프로젝트 이름이 맞는지
3. 이전 배포 설정과 현재 배포 설정이 충돌하지 않는지
4. 환경변수나 빌드 설정이 Cloudflare 환경에 제대로 반영되어 있는지
5. 로컬에서 성공한 빌드가 Cloudflare 환경에서도 동일하게 수행되는지

처음에는 재배포를 반복하며 해결하려 했지만, 이후에는 재배포보다 로그를 기준으로 원인을 좁히는 방식으로 접근했다.

결과

Cloudflare 배포 오류를 단순한 “실패”로 보지 않고,
빌드 단계 문제 / Cloudflare 설정 문제 / Worker 버전 생성 문제 / 배포 후 연결 문제로 나누어 확인하는 기준을 만들 수 있었다.

이 경험을 통해 배포 과정에서는 에러 메시지를 그대로 읽는 것보다, 해당 에러가 발생한 계층을 먼저 구분하는 것이 중요하다는 점을 배웠다.

4. 마주한 문제 3: 프론트에서는 API 요청이 실패했지만 원인을 한 번에 특정하기 어려웠다

문제 상황

배포 후 프론트 화면에서 API 요청이 정상적으로 동작하지 않았다. 브라우저 콘솔에는 **ERR_FAILED**와 같은 네트워크 오류가 표시되었고, 일부 화면에서는 응답이 비어 있거나 비동기 응답 관련 오류가 발생했다.

처음에는 프론트 코드 자체의 문제라고 생각했지만, 실제로는 API 요청 주소, 백엔드 서버 접근 가능 여부, CORS 설정, 환경변수, 로컬 주소 사용 여부 등 여러 가능성을 함께 봐야 했다.

특히 로컬 개발 환경에서는 **localhost** 또는 **127.0.0.1** 주소로 API를 호출해도 문제가 없었지만, 배포 환경에서는 사용자의 브라우저가 해당 로컬 주소에 접근할 수 없기 때문에 API base URL 설정을 다시 확인해야 했다.

어떻게 해결했는가

브라우저 개발자도구의 **Console**과 **Network** 탭을 기준으로 요청 흐름을 추적했다. 단순히 “API가 안 된다”고 판단하지 않고, 다음 순서로 확인했다.

1. 프론트에서 실제로 어떤 URL로 요청을 보내고 있는지
2. 요청이 브라우저에서 차단되는지, 서버까지 도달하는지

3. 응답 상태 코드가 200, 400, 401, 500, 502 중 무엇인지
4. 응답 본문이 비어 있는지, 에러 메시지를 포함하는지
5. 배포 환경에서 사용하는 API base URL이 로컬 주소로 남아 있지 않은지
6. 백엔드의 CORS, ALLOWED_HOSTS, CSRF 관련 설정이 배포 도메인을 허용하는지

이를 통해 프론트 화면에서 보이는 오류만으로 판단하지 않고, 실제 요청과 응답의 흐름을 기준으로 문제를 좁혀갔다.

결과

API 오류를 프론트 문제로만 보지 않고,
프론트 요청 URL → 브라우저 네트워크 요청 → 백엔드 응답 → 환경변수 및 CORS
설정의 흐름으로 분석할 수 있게 되었다.

또한 배포 환경에서는 로컬과 달리 프론트와 백엔드가 서로 다른 출처에서 동작하기 때문에,
API 연결 설정을 명확히 관리해야 한다는 점을 배웠다.

5. 마주한 문제 4: 로컬 환경과 배포 환경의 차이를 충분히 고려하지 못했다

문제 상황

로컬에서는 프론트엔드와 백엔드가 각각 정상적으로 동작했기 때문에, 배포 후에도 큰 문제 없이 연결될 것이라고 예상했다. 하지만 배포 환경에서는 로컬에서 암묵적으로 처리되던 설정들이 직접적인 장애 요인이 되었다.

예를 들어 로컬 개발 환경에서는 프론트가 localhost:5173에서 실행되고 백엔드가 127.0.0.1:8000에서 실행되기 때문에 요청 흐름을 쉽게 확인할 수 있었다. 그러나 실제 배포 환경에서는 사용자의 브라우저가 로컬 백엔드 주소에 접근할 수 없고, Cloudflare에 올라간 프론트가 어떤 백엔드 주소를 바라보는지도 명확히 설정해야 했다.

어떻게 해결했는가

로컬 기준으로 작성된 설정과 배포 기준으로 필요한 설정을 분리했다.
특히 다음 항목을 중심으로 점검했다.

- 프론트의 API base URL이 배포 환경용 주소를 바라보는지
- 백엔드가 외부 요청을 받을 수 있도록 host와 origin 설정이 되어 있는지
- CORS 설정에 배포된 프론트 도메인이 포함되어 있는지
- .env 파일의 값이 로컬용과 배포용으로 분리되어 있는지
- 배포 후 변경된 환경변수가 실제 빌드에 반영되었는지

이를 통해 로컬에서 동작하는 코드와 배포 환경에서 동작하는 서비스는 같은 코드라도 다른 조건에서 실행된다는 점을 기준으로 문제를 점검했다.

결과

배포 환경에서는 코드 자체보다 설정 차이로 문제가 발생하는 경우가 많다는 것을 이해했다. 또한 로컬 개발용 설정과 운영/배포용 설정을 분리해 관리해야 하며, 특히 **API URL**, **CORS**, **host** 설정은 배포 전에 반드시 확인해야 하는 항목이라는 기준을 세울 수 있었다.

6. 마주한 문제 5: 무엇부터 확인해야 하는지 기준이 없어 시간이 오래 걸렸다

문제 상황

배포 오류를 마주했을 때 가장 어려웠던 점은 특정 기술 하나가 아니라, “어디부터 확인해야 하는가”였다.

브라우저 콘솔, **Network** 탭, **Cloudflare** 배포 로그, 환경변수, 백엔드 서버 상태, **API** 엔드포인트 등 확인해야 할 지점이 많았고, 처음에는 그 순서가 정리되어 있지 않았다.

그 결과 같은 문제를 두고도 재배포를 반복하거나, 프론트 코드를 먼저 의심했다가 다시 배포 설정으로 돌아가는 식의 비효율이 발생했다.

어떻게 해결했는가

문제를 확인하는 순서를 체크리스트로 정리했다.

이후에는 오류가 발생했을 때 무작정 코드를 수정하거나 재배포하지 않고, 아래 순서대로 점검하는 방식을 적용했다.

1. 배포 로그에서 빌드와 업로드가 정상적으로 끝났는지 확인
2. **Cloudflare**에서 제공하는 기본 **URL**로 접속 가능한지 확인
3. 외부 공개 **URL** 또는 **route/custom domain**이 올바르게 연결되어 있는지 확인
4. 프론트에서 **API** 요청이 어느 주소로 나가는지 **Network** 탭에서 확인
5. **API** 요청이 백엔드까지 도달하는지 확인
6. 응답 상태 코드와 에러 메시지를 확인
7. 환경변수, **CORS**, **host** 설정을 확인
8. 필요한 경우에만 코드를 수정하고 재배포

결과

문제를 감으로 해결하는 것이 아니라, 요청 흐름과 배포 구조를 기준으로 순서대로 좁혀가는 디버깅 방식을 익혔다.

이 경험을 통해 배포 작업에서는 “무엇이 안 되는가”보다 “어디까지는 정상이고, 어디서부터 실패하는가”를 구분하는 것이 중요하다는 점을 배웠다.

7. 최종 결과

이번 배포 과정을 통해 단순히 **Cloudflare**에 프로젝트를 올리는 것을 넘어, 실제 서비스가 동작하기 위해 필요한 연결 구조를 이해하게 되었다.

최종적으로 얻은 결과는 다음과 같다.

- 배포 성공과 서비스 접근 가능 상태를 분리해서 판단할 수 있게 되었다.
- **Cloudflare**의 기본 **URL**, 외부 공개 **URL**, **route/custom domain**의 역할을 구분할 수 있게 되었다.
- 프론트엔드에서 발생한 **API** 오류를 **Console**과 **Network** 탭을 통해 추적하는 방법을 익혔다.
- 로컬 환경과 배포 환경의 차이를 고려해 환경변수와 **API base URL**을 점검하는 기준을 만들었다.
- **CORS**, **host**, **endpoint** 설정이 프론트-백엔드 연결에 직접적인 영향을 준다는 점을 이해했다.
- 이후 배포 작업에서는 배포 아키텍처와 점검 체크리스트를 먼저 문서화해야 한다는 기준을 세웠다.

8. 배운 점과 느낀 점

이번 경험을 통해 배포는 단순히 코드를 서버나 플랫폼에 올리는 작업이 아니라는 것을 배웠다. 배포는 프론트엔드, 백엔드, 도메인, 환경변수, **API** 요청 흐름, 권한, 라우팅 설정이 모두 맞물려야 완성되는 통합 작업이었다.

특히 로컬에서는 정상적으로 동작하던 기능도 배포 환경에서는 쉽게 실패할 수 있다는 점이 인상 깊었다. 로컬 환경에서는 개발자가 직접 실행하고 있는 주소와 포트를 알고 있기 때문에 문제를 단순하게 볼 수 있지만, 배포 환경에서는 사용자가 접근하는 **URL**, 프론트가 호출하는 **API** 주소, 백엔드가 허용하는 **origin**, **Cloudflare**가 연결하는 **route**가 모두 맞아야 했다.

처음에는 오류가 발생했을 때 “왜 안 되지?”라는 생각이 먼저 들었지만, 문제를 해결하는 과정에서 “어디까지는 정상이고, 어디서부터 실패하는가?”를 확인하는 방식이 더 중요하다는 것을 알게 되었다. 배포 로그, 브라우저 콘솔, **Network** 탭, 환경변수, 백엔드 설정을 순서대로 확인하면서 문제를 구조적으로 좁혀가는 경험을 할 수 있었다.

또한 이번 경험은 개발 과정에서 운영 관점이 왜 중요한지 느끼게 해주었다. 기능 구현만으로는 서비스가 완성되지 않으며, 사용자가 실제로 접근할 수 있고 안정적으로 요청을 주고받을 수 있어야 비로소 서비스로 동작한다는 점을 배웠다.

따라서 이 경험은 단순한 배포 실패나 시행착오가 아니라, 프론트엔드와 백엔드가 실제 서비스 환경에서 어떻게 연결되는지 이해하고, 문제 발생 시 원인을 분해해 확인하는 역량을 키운 과정이었다. 이후에는 배포 전 체크리스트, 환경변수 관리, **API** 연결 구조, 도메인 및 라우팅 구조를 먼저 문서화하여 같은 문제를 반복하지 않도록 개선하고자 한다.

9. 요약

Cloudflare를 활용해 **Lumière** 프로젝트를 배포하는 과정에서, 단순히 코드 배포가 완료되는 것과 실제 사용자가 접근 가능한 서비스 상태는 다르다는 점을 경험했다. 배포 후 외부 **URL** 구조, **route/custom domain** 설정, **API** 요청 실패, 로컬 환경변수 잔존, **CORS** 및 백엔드 연결 문제를 마주했고, 이를 배포 로그와 브라우저 **Network** 탭을 기준으로 하나씩 분리해 확인했다.

이 과정에서 프론트엔드와 백엔드가 각각 정상적으로 동작하더라도, 배포 환경의 도메인·환경변수·**API endpoint**·라우팅 설정이 맞지 않으면 서비스 전체가 정상적으로 동작하지 않을 수 있다는 점을 배웠다. 이후 배포 문제를 해결할 때는 무작정 재배포하기보다, 배포 로그 → 공개 **URL** → 라우팅 → 환경변수 → **API** 요청 → 백엔드 응답 순서로 확인하는 체크리스트를 정리했다.

이를 통해 배포를 단순 실행 작업이 아니라 서비스 운영을 위한 연결 구조 설계 과정으로 이해하게 되었고, 문제를 계층별로 분해해 원인을 좁혀가는 트러블슈팅 역량을 기를 수 있었다.

배포 트러블슈팅 회고

Cloudflare 배포 과정에서 마주한 문제를 HOW TO → RESULT 구조로 정리



1 상황 요약

- 목표: 프론트/백엔드를 외부에서 접속 가능한 형태로 배포하고, 실제 서비스처럼 동작하도록 연결하기
- 문제: 배포는 되었지만 링크 노출 방식, API 연결, 설정 반영 여부를 확인하는 과정에서 여러 혼란이 발생
- 핵심 포인트: 단순한 '배포 성공/실패'가 아니라, 도메인·환경변수·라우팅·API 요청 흐름을 함께 이해해야 했음

2 마주한 문제들

1



배포는 되었는데 외부에 노출할 링크 구조를 이해하기 어려웠다

workers.dev 주소와 실제 외부에 연결할 주소가 어떻게 다른지, 어디에서 route / custom domain을 설정하는지 헷갈렸다.

2



Cloudflare 배포 중 API/버전 생성 오류가 발생했다

배포 로그에는 완료되지 않는 메시지가 보였고, 계정·스크립트·배포 설정 중 어디가 문제인지 바로 파악하기 어려웠다.

3



프론트에서 API 요청이 실패했지만 원인을 한 번에 특정하기 어려웠다

ERR_FAILED, 비동기 응답 예외, 빈 응답 화면 등 증상은 보였지만, 네트워크 문제인지 환경변수 문제인지, 백엔드 연결 문제인지 구분이 필요했다.

4



'어디를 확인해야 하는가' 자체가 가장 큰 어려움이였다

브라우저 콘솔, Network 탭, 배포 로그, 환경변수, 엔드포인트 설정을 어떤 순서로 봐야 할지 처음에는 체계가 없었다.

3 HOW TO

STEP 1



배포 상태부터 분리해서 확인

- 배포 자체가 실패한 것인지, 배포는 됐지만 연결이 안 되는 것인지 먼저 구분했다.
- 배포 로그와 제공된 기본 URL을 확인해 '실행 여부'와 '접속 여부'를 분리해서 판단했다.

STEP 2



도메인/라우팅 구조 이해

- workers.dev 기본 주소와 외부 공개용 route 또는 custom domain의 역할을 정리했다.
- 사용자가 접속하는 주소와 실제 서비스가 연결되는 경로를 구분해서 이해하려고 했다.

STEP 3



API 요청 흐름 추적

- 브라우저 개발자도구의 Console / Network 탭을 통해 어떤 요청이 어느 URL로 나가고, 어떤 상태코드로 실패하는지 확인했다.
- 중앙만 보지 않고 '요청 → 응답 → 응대' 메시지의 흐름으로 문제를 좁혀 갔다.

STEP 4



환경변수와 엔드포인트 점검

- 로컬 주소(127.0.0.1 등)가 배포 환경에 그대로 남아 있지 않은지, 프론트의 API base URL이 올바르게 연결되는지 점검했다.
- 배포 환경에서는 코드보다 설정 차이로 문제가 나는 경우가 많다는 점을 체감했다.

STEP 5



재배포보다 체크리스트화

- 문제가 생길 때마다 무작정 재배포하기보다, 확인해야 할 항목을 순서화했다.
- 배포 로그 → 도메인/라우팅 → 환경변수 → 네트워크 요청 → 실제 API 응답 순으로 확인하는 기준을 만들었다.

4 RESULT



- ✓ 배포 문제를 '코드 문제'가 아니라 '설정 + 연결 구조 문제'로 바라보는 시각을 갖게 되었다.
- ✓ 공개 URL, 라우팅, API 요청 흐름을 구분해 확인하는 방법을 익혔다.
- ✓ 프론트와 백엔드가 각각 정상이어도, 연결 설정이 맞지 않으면 서비스는 동작하지 않는다는 점을 명확히 이해했다.
- ✓ 이 경험을 통해 이후에는 배포 아키텍처와 점검 체크리스트를 먼저 문서화해야겠다는 기준이 생겼다.

5 배운 점과 느낀 점



- 배포는 단순히 코드를 올리는 과정이 아니라, 도메인·권한·환경변수·라우팅·API 연결을 함께 맞추는 통합 작업이라는 것을 배웠다.
- 문제가 생겼을 때 '왜 안 되지?'에 머무르기보다, 어디서부터 확인해야 하는지를 구조적으로 정리하는 습관이 중요하다는 점을 느꼈다.
- 특히 사용자가 보는 프론트 화면 뒤에는 수많은 연결 지점이 있고, 그중 하나만 어긋나도 전체 서비스 경험이 무너질 수 있다는 점이 인상 깊었다.
- 이번 경험은 기술 자체보다도, 문제를 분해하고 확인 순서를 세우는 디버깅 사고방식을 키워준 과정이었다.
- 포트폴리오에서는 이 경험을 '배포 성공' 한 줄로 끝내기보다, 시행착오를 통해 배포 구조를 이해하고 운영 관점의 시야를 넓힌 사례로 정리할 수 있다.